

Below is a comprehensive table of the **parameters/criteria you can use to compare two Collection classes in Java**. The comparison should consider not only whether they both implement Collection, but also their ordering, duplicates, null handling, performance, thread safety, mutability, memory usage, and typical use cases.

1. Core Parameters for Comparing Java Collection Classes

#	Comparison Parameter	Definition	Questions to Ask	Example
1	Interface	The interface implemented by the class.	Does it implement List, Set, Queue, Deque, or Map?	ArrayList implements List
2	Collection Hierarchy	The position of the class in the Java Collections Framework hierarchy.	What interfaces/classes does it extend or implement?	ArrayList → AbstractList → List → Collection
3	Ordering	Determines whether elements maintain a predictable order.	Is insertion order, sorted order, or no guaranteed order maintained?	ArrayList maintains insertion order
4	Insertion Order	Whether elements appear in the same order in which they were inserted.	Does add(A); add(B); add(C) produce A,B,C?	LinkedHashSet maintains insertion order
5	Sorted Order	Whether elements are automatically maintained according to a sorting rule.	Does the collection automatically sort elements?	TreeSet maintains sorted order
6	Duplicates	Determines whether duplicate elements are permitted.	Can the same element occur more than once?	ArrayList → Yes; HashSet → No
7	Null Elements	Determines whether null can be stored.	Can one or more null values be inserted?	ArrayList permits multiple nulls

8	Null Keys/Values	Relevant mainly to Map implementations.	Can keys/values be null?	HashMap permits one null key
9	Index-Based Access	Whether elements can be accessed directly using an index.	Can you use get(index)?	ArrayList.get(2)
10	Random Access	Efficiency of accessing an arbitrary element.	Is get(i) approximately $O(1)$?	ArrayList → Yes
11	Sequential Access	Whether elements are primarily accessed sequentially.	Is traversal more efficient than random access?	LinkedList
12	Search Performance	Time required to find an element.	How efficient are contains() or key lookup operations?	HashSet.contains() → average $O(1)$
13	Insertion Performance	Time required to insert elements.	How efficient is add()?	ArrayList.add() → amortized $O(1)$
14	Deletion Performance	Time required to remove elements.	How efficient is remove()?	HashSet.remove() → average $O(1)$
15	Access Performance	Time required to retrieve an element.	Is retrieval constant, logarithmic, or linear?	TreeMap.get() → $O(\log n)$
16	Sorting Performance	Cost of sorting elements.	Is sorting automatic or performed separately?	TreeSet maintains sorted order
17	Iteration Performance	Efficiency of traversing elements.	How fast is iteration?	ArrayList generally has efficient iteration
18	Time Complexity	Big-O performance of major operations.	What are complexities of add, get, remove, contains?	ArrayList.get() → $O(1)$

19	Space Complexity	Additional memory required by the implementation.	Does the implementation require extra structures?	LinkedList requires node objects
20	Memory Overhead	Extra memory consumed by internal data structures.	Does each element require additional references/nodes?	LinkedList has node overhead
21	Underlying Data Structure	Internal structure used to store elements.	Array, linked nodes, hash table, tree, etc.?	ArrayList → dynamic array
22	Capacity	Internal storage capacity.	Does the collection maintain unused capacity?	ArrayList has internal array capacity
23	Capacity Growth	How internal storage grows when capacity is exceeded.	How does the collection resize?	ArrayList grows its backing array
24	Load Factor	Determines when a hash-based structure should resize.	What load factor controls resizing?	HashMap default load factor is 0.75
25	Initial Capacity	Initial internal storage size.	Can initial capacity be specified?	<code>new ArrayList<>(100)</code>
26	Hashing	Use of hash codes to locate elements.	Does the implementation use <code>hashCode()</code> ?	HashSet, HashMap
27	Equality Mechanism	Determines how elements are considered equal.	Does it use <code>equals()</code> or identity?	HashSet uses <code>hashCode()</code> + <code>equals()</code>
28	Comparison Mechanism	Determines how elements are ordered.	Does it use Comparable or Comparator?	TreeSet
29	Comparable Support	Natural ordering provided by elements themselves.	Do elements implement Comparable?	String implements Comparable
30	Comparator Support	External/custom ordering.	Can a custom Comparator be supplied?	<code>new TreeSet<>(Comparator.reverseOrd</code>

				er()
--	--	--	--	------

2. Behavioral Parameters

#	Parameter	Definition	Example / Explanation
31	Mutability	Whether the collection can be modified after creation.	ArrayList is mutable
32	Immutability	Whether the collection cannot be structurally modified.	List.of("A","B") is unmodifiable
33	Unmodifiable View	A view that prevents modification through that reference but may reflect changes in the backing collection.	Collections.unmodifiableList(list)
34	Structural Modification	Operations that change collection size/structure.	add(), remove(), clear()
35	Element Modification	Modification of objects stored inside the collection.	A mutable object inside an unmodifiable list may still be changed
36	Thread Safety	Whether multiple threads can safely access/modify the collection.	Vector provides synchronized methods
37	Concurrent Access	Ability to safely operate under concurrent modification/access.	ConcurrentHashMap
38	Synchronization	Whether operations are synchronized.	Collections.synchronizedList()
39	Fail-Fast Behavior	Iterators may detect structural modification and throw ConcurrentModificationException.	Typical for many standard collections

40	Weakly Consistent Iteration	Concurrent collections may allow changes while iteration proceeds without fail-fast behavior.	ConcurrentHashMap
41	Iterator Type	Type of traversal mechanism supported.	Iterator, ListIterator, Spliterator
42	Iterator Modification	Whether iterator supports remove() or list-specific operations.	ListIterator supports add() and set()
43	Spliterator Characteristics	Properties exposed for stream processing.	ORDERED, DISTINCT, SORTED, SIZED
44	Stream Support	Whether it can directly produce sequential/parallel streams.	All Collection implementations provide stream()
45	Parallel Processing	Suitability for parallel stream processing.	Depends on splitting/traversal characteristics
46	Serialization	Whether instances can be serialized.	Depends on implementation and element types
47	Cloneability	Whether the implementation supports cloning.	ArrayList implements Cloneable
48	Defensive Copy Requirement	Whether a copy should be made to prevent external modification.	Important when exposing mutable collections
49	View Support	Whether the implementation provides views of data.	Map.keySet(), Map.values(), Map.entrySet()
50	Backed View	Whether a returned collection view reflects changes in the original collection.	map.keySet() is backed by the map

3. List-Specific Comparison Parameters

Parameter	ArrayList	LinkedList	Meaning
Data structure	Dynamic array	Doubly linked list	Internal storage
Index access	Fast	Slow	get(index)
get()	$O(1)$	$O(n)$	Random access
Append	Amortized $O(1)$	$O(1)$	Add at end
Insert at beginning	$O(n)$	$O(1)$	Beginning insertion
Remove beginning	$O(n)$	$O(1)$	Beginning removal
Insert middle	$O(n)$	$O(n)$ to locate + $O(1)$ link change	Depends on locating position
Duplicates	Yes	Yes	Duplicate elements
null	Yes	Yes	Null elements
Insertion order	Yes	Yes	Element ordering
ListIterator	Yes	Yes	Bidirectional iteration
Deque operations	No	Yes	LinkedList also implements Deque
Memory overhead	Lower	Higher	Linked nodes require additional references

Example:

```
List<String> list = new ArrayList<>();
```

```
list.add("Java");  
list.add("Spring");  
list.add("Java");
```

```
System.out.println(list);  
// [Java, Spring, Java]
```

```
System.out.println(list.get(1));  
// Spring
```

4. Set-Specific Comparison Parameters

Parameter	HashSet	LinkedHashSet	TreeSet
Duplicates	No	No	No
Insertion order	No guaranteed order	Yes	No
Sorted order	No	No	Yes
Internal structure	Hash table	Hash table + linked structure	Red-black tree
Average add()	O(1)	O(1)	O(log n)
Average contains()	O(1)	O(1)	O(log n)
null	One null generally permitted	One null generally permitted	Generally no null
Comparable	Not required	Not required	Typically required for natural ordering

Comparator	No	No	Yes
Best use	Fast uniqueness	Uniqueness + insertion order	Sorted unique elements

Example:

```
Set<Integer> set = new TreeSet<>();
```

```
set.add(30);
set.add(10);
set.add(20);
set.add(10);
```

```
System.out.println(set);
// [10, 20, 30]
```

The duplicate 10 is removed and elements are sorted.

5. Queue and Deque Comparison Parameters

Parameter	PriorityQueue	ArrayDeque	LinkedList
Primary purpose	Priority-based processing	Queue/stack	List + queue/deque
Ordering	Priority order	Insertion order at ends	Insertion order
FIFO	Not necessarily	Yes	Yes
LIFO	No	Yes	Yes
null	No	No	Yes

Duplicates	Yes	Yes	Yes
Random access	No	No	Yes, but inefficient
offer()	Yes	Yes	Yes
poll()	Priority element	First element	First element
peek()	Highest-priority element	First element	First element

Example:

```
Queue<Integer> queue = new PriorityQueue<>();
```

```
queue.offer(30);
queue.offer(10);
queue.offer(20);
```

```
System.out.println(queue.poll());
// 10
```

6. Map Comparison Parameters

Map is part of the Java Collections Framework, but **it does not extend Collection**. Therefore, when comparing collection classes broadly, Map implementations should be compared separately.

Parameter	HashMap	LinkedHashMap	TreeMap
Key duplicates	No	No	No
Value duplicates	Yes	Yes	Yes

Key ordering	No guaranteed order	Insertion/access order	Sorted
Data structure	Hash table	Hash table + linked list	Red-black tree
Average put()	O(1)	O(1)	O(log n)
Average get()	O(1)	O(1)	O(log n)
Average remove()	O(1)	O(1)	O(log n)
Null key	One	One	Generally not supported
Null values	Yes	Yes	Yes
Comparator	No	No	Yes
Best use	Fast key/value lookup	Predictable iteration	Sorted keys

Example:

```
Map<Integer, String> map = new HashMap<>();
```

```
map.put(101, "Anurag");
map.put(102, "Rahul");
map.put(103, "Amit");
```

```
System.out.println(map.get(102));
// Rahul
```

7. Equality and Comparison Parameters

One of the most important areas when comparing two collection objects is **what equals()** actually means for that collection type.

Collection Type	Equality Concept	Example
List	Same elements in the same order	[A, B] equals [A, B]
Set	Same unique elements, regardless of iteration order	{A, B} equals {B, A}
Map	Same key-value mappings	{1=A, 2=B} equals {2=B, 1=A}
Queue	Depends on the specific implementation/type and its equality contract	Check implementation contract
Custom Collection	Depends on overridden equals()	Developer-defined

Example:

```
List<Integer> list1 = List.of(10, 20, 30);
List<Integer> list2 = List.of(10, 20, 30);
```

```
System.out.println(list1.equals(list2));
// true
```

But:

```
List<Integer> list1 = List.of(10, 20, 30);
List<Integer> list2 = List.of(30, 20, 10);
```

```
System.out.println(list1.equals(list2));
// false
```

Because List.equals() is order-sensitive.

8. Performance Parameters

Operation	ArrayList	LinkedList	HashSet	TreeSet	HashMap	TreeMap
get()	$O(1)$	$O(n)$	N/A	N/A	$O(1)^*$	$O(\log n)$
add()	$O(1)^*$	$O(1)$ at end	$O(1)^*$	$O(\log n)$	N/A	N/A
contains()	$O(n)$	$O(n)$	$O(1)^*$	$O(\log n)$	$O(1)^*$ key	$O(\log n)$
remove()	$O(n)$	$O(n)$	$O(1)^*$	$O(\log n)$	$O(1)^*$ key	$O(\log n)$
Sorted insertion	N/A	N/A	N/A	$O(\log n)$	N/A	$O(\log n)$

* = average/amortized complexity under normal assumptions.

Actual performance can vary depending on hashing quality, resizing, data distribution, implementation details, and workload.

9. Null, Duplicate and Ordering Matrix

Class	Duplicates	null	Insertion Order	Sorted Order
ArrayList	✓ Yes	✓ Yes	✓ Yes	✗ No
LinkedList	✓ Yes	✓ Yes	✓ Yes	✗ No
HashSet	✗ No	✓ One generally	✗ No guarantee	✗ No
LinkedHashSet	✗ No	✓ One generally	✓ Yes	✗ No
TreeSet	✗ No	✗ Generally no	✗ No	✓ Yes

PriorityQueue	✓ Yes	✗ No	✗ No	Priority-based
ArrayDeque	✓ Yes	✗ No	✓ Yes	✗ No
HashMap keys	✗ No	✓ One	✗ No guarantee	✗ No
LinkedHashMap keys	✗ No	✓ One	✓ Yes	✗ No
TreeMap keys	✗ No	✗ Generally no	✗ No	✓ Yes

10. Thread-Safety Comparison

Collection	Thread-Safe?	Recommended Concurrent Alternative
ArrayList	✗ No	CopyOnWriteArrayList
LinkedList	✗ No	Usually use an appropriate concurrent queue/deque
HashSet	✗ No	ConcurrentHashMap.newKeySet()
LinkedHashSet	✗ No	No direct equivalent; synchronize or redesign
TreeSet	✗ No	ConcurrentSkipListSet
HashMap	✗ No	ConcurrentHashMap
LinkedHashMap	✗ No	Synchronization/custom design
TreeMap	✗ No	ConcurrentSkipListMap
Vector	✓ Synchronized methods	Usually prefer modern concurrent collections

Hashtable	✓ Synchronized methods	ConcurrentHashMap
-----------	------------------------	-------------------

11. API/Functional Comparison Parameters

Parameter	What to Compare	Example
add()	How elements are inserted	list.add("Java")
addAll()	Bulk insertion	list.addAll(other)
remove()	Element deletion	list.remove("Java")
removeIf()	Predicate-based deletion	list.removeIf(x -> x.startsWith("J"))
contains()	Element existence	list.contains("Java")
containsAll()	Multiple-element existence	list.containsAll(other)
size()	Number of elements	list.size()
isEmpty()	Whether collection is empty	list.isEmpty()
clear()	Removes all elements	list.clear()
iterator()	Sequential traversal	collection.iterator()
spliterator()	Advanced traversal/parallel processing	collection.spliterator()
stream()	Sequential stream	collection.stream()
parallelStream()	Parallel stream	collection.parallelStream()
forEach()	Element processing	list.forEach(System.out::println)

<code>toArray()</code>	Convert to array	<code>list.toArray()</code>

12. Best Parameters to Use in a Real-World Comparison

When comparing **any two Java collection classes**, this is a useful checklist:

Priority	Parameter	Why It Matters
1	Interface	Determines the fundamental behavior
2	Ordering	Determines iteration order
3	Duplicates	Determines whether repeated values are allowed
4	null support	Important for data validation
5	Underlying data structure	Explains behavior and performance
6	<code>add()</code> complexity	Important for insertion-heavy applications
7	<code>get()</code> complexity	Important for read/access-heavy applications
8	<code>remove()</code> complexity	Important for deletion-heavy applications
9	<code>contains()</code> complexity	Important for search-heavy applications
10	Sorting	Important when ordered data is required
11	Memory consumption	Important for large datasets
12	Thread safety	Critical in multithreaded applications

13	Mutability	Important for API and application design
14	Iterator behavior	Important during traversal/modification
15	Stream support	Important for functional programming
16	Concurrent alternatives	Important for multithreaded systems
17	Equality semantics	Important when comparing collections
18	Typical use case	Helps select the right implementation
19	Scalability	Important for large data volumes
20	Java version/API availability	Important when maintaining compatibility

Example: How to compare ArrayList vs HashSet

Parameter	ArrayList	HashSet	Winner/Use Case
Interface	List	Set	Depends
Duplicates	Allowed	Not allowed	ArrayList for duplicates
Insertion order	Maintained	Not guaranteed	ArrayList
Index access	Yes	No	ArrayList
get(index)	O(1)	N/A	ArrayList
contains()	O(n)	O(1) average	HashSet
remove()	O(n)	O(1) average	HashSet

Null	Allowed	One generally	Depends
Sorting	Manual	Manual	Depends
Memory	Generally lower per element	Hash-table overhead	ArrayList
Uniqueness	No	Yes	HashSet
Best use	Ordered/duplicate data	Fast unique lookup	Depends

Key rule: There is no universally "best" Collection class. Choose based on the required **ordering + duplicate policy + null policy + access pattern + time complexity + memory requirements + thread-safety requirements**.